Introduction to Database Management Systems

Md. Tariquzzaman
Junior Lecturer, Dept. of Computer Science and Engineering
Islamic University of Technology (IUT)

CSE 4307: Database Management Systems

Contents

The Kingdom of Kod
History
Introduction to Database Management Systems
Database-System Applications
Basic Concepts
Purpose of Database Systems
View of Data
Schemas and Instances
Database Languages
Database Design
Database Engine
Database Architecture
Database Users and Administrator

The Kingdom of Kod

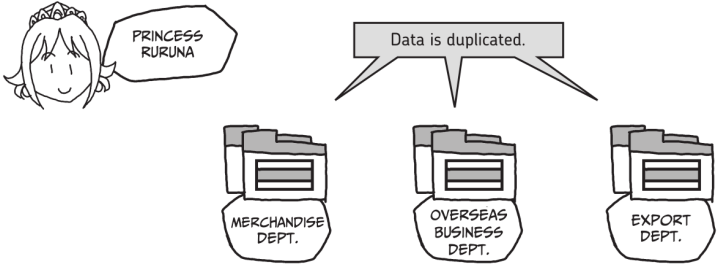
What is up in the Kingdom of Kod?

The Kingdom currently has three departments: the Merchandise Department, the Overseas Business Department, and the Export Department.

The Merchandise Department keeps track of all fruit produced in the country, the Overseas Business Department manages the foreign countries that are the Kingdom's business partners, and the Export Department keeps records of the amount of fruit the Kingdom exports.

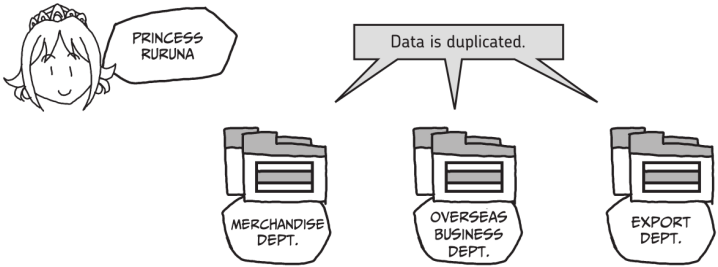
Data Is Duplicated

Each department must enter the data, store the data, then print receipts for confirmation, all of which is a waste. In addition, data trapped in one particular department is never shared effectively with the other departments. The system also creates problems when someone needs to change the data.



Data Is Duplicated

For example, let's assume that the price of apples changes. To deal with this, Princess Ruruna must notify every department individually that the price of apples has changed. Isn't that inconvenient?



Data Can Conflict

It may seem easy enough to notify each department that the price of apples has changed, but it can create a new set of problems. Let's say that Princess Ruruna does notify the three departments that the price of apples has changed.

MERCHANDISE DEPARTMENT

Product name	Unit price	
Melon	800G	
Strawberry	150G	
Apple	*120G*	
Lemon	200G	

OVERSEAS BUSINESS DEPARTMENT

Product name	Unit price	
Melon	800G	
Strawberry	150G	
Apple	*100G*	
Lemon	200G	

EXPORT DEPARTMENT

Product name	Unit price	
Melon	800G	
Strawberry	150G	
Apple	*300G*	
Lemon	200G	

Data Can Conflict

However, the Overseas Business Department may forget to change the price, or the Export Department might change the price to 300G instead of 120G.

MERCHANDISE DEPARTMENT

Product name	Unit price	
Melon	800G	
Strawberry	150G	
Apple	*120G*	
Lemon	200G	

OVERSEAS BUSINESS DEPARTMENT

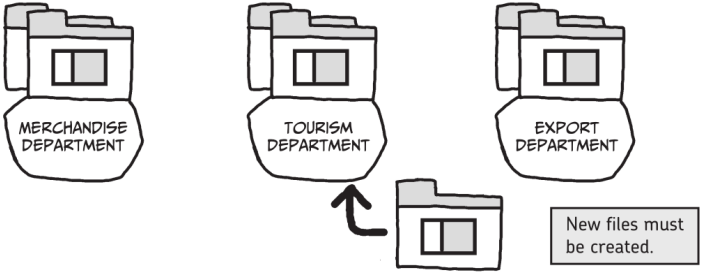
Product name	Unit price	
Melon	800G	
Strawberry	150G	
Apple	*100G*	
Lemon	200G	

EXPORT DEPARTMENT

Product name	Unit price	
Melon	800G	
Strawberry	150G	
Apple	*300G*	
Lemon	200G	

Data Is Difficult to Update

For example, let's say that the King wants to launch a new Tourism Department.



FILE FOR MERCHANDISE DEPT.

Product name	Unit price	
Melon	800G	
Strawberry	150G	
Apple	120G	
Lemon	200G	

FILE FOR TOURISM DEPT.

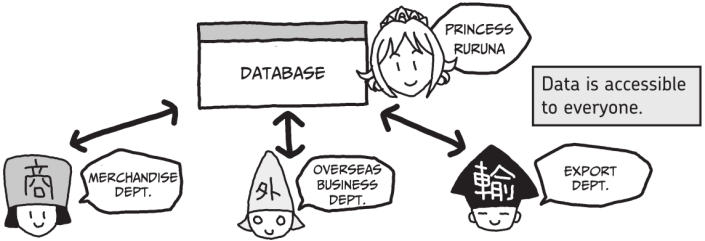
Product name	Unit price	
Melon	*800G*	
Strawberry	*150G*	
Apple	*120G*	
Lemon	*200G*	

Data Is Difficult to Update

This, in turn, increases the amount of duplicated data created when a new department starts. Considering these weaknesses, the current system is not efficient. It makes it difficult to start new projects and respond to environmental changes.

The Solution - Database!

Uniform data management ensures that each department has the correct information, because each department sends a query to a single source of data.



It prevents data conflicts, and it also eliminates duplicated data, allowing for easy introduction and integration of new departments.

History

Evolution of Database Systems

- **1950s–1960s: File-based systems** — data stored in flat files on magnetic tapes; no standard query language or data independence.
- **1970s: Relational Model (E. F. Codd)** — introduced tables, keys, and relationships; foundation of modern DBMS.
- **1980s: Commercial DBMS & SQL Standardization** — Oracle, IBM DB2, and others popularized SQL as the universal query language.
- **1990s: Data Warehousing & Web Databases** — focus shifted to large-scale analytics and online access.
- **2000s: NoSQL & Big Data Era** — need for scalability and unstructured data led to systems like MongoDB, Cassandra, and Hadoop.
- **2010s–Present: Cloud & In-Memory Databases** — real-time analytics, global accessibility, and high-speed processing (e.g., AWS RDS, Redis, Snowflake).

What is a Database Management System?

- A **Database Management System (DBMS)** is a specialized software system designed to store, organize, and manage large amounts of data efficiently.
- It acts as an interface between the user and the database — allowing users and applications to **create**, **retrieve**, **update**, and **delete** data (often called **CRUD** operations).
- A DBMS ensures that data is **consistent**, **secure**, and **easily accessible**, even when many users access it simultaneously.

Introduction to Database Management Systems

What is a Database Management System?

- Common examples include **MySQL**, **PostgreSQL**, **Oracle Database**, and **MongoDB**.
- Think of a DBMS as a **librarian** — instead of searching through thousands of scattered papers (files), you simply ask the librarian (DBMS), who knows exactly where and how to find the information.

What is a Database Management System?

What is a Database Management System?

Database-System Applications

Examples of Database Applications

- **Enterprise:** Sales, HR, Accounting
- **Banking:** Accounts, Loans, Transactions
- **Universities:** Students, Courses, Grades
- **Airlines:** Reservations, Schedules
- **Telecom:** Calls, Data Usage, Billing
- **E-commerce:** Orders, Recommendations
- **Navigation:** Maps, Routes, Locations

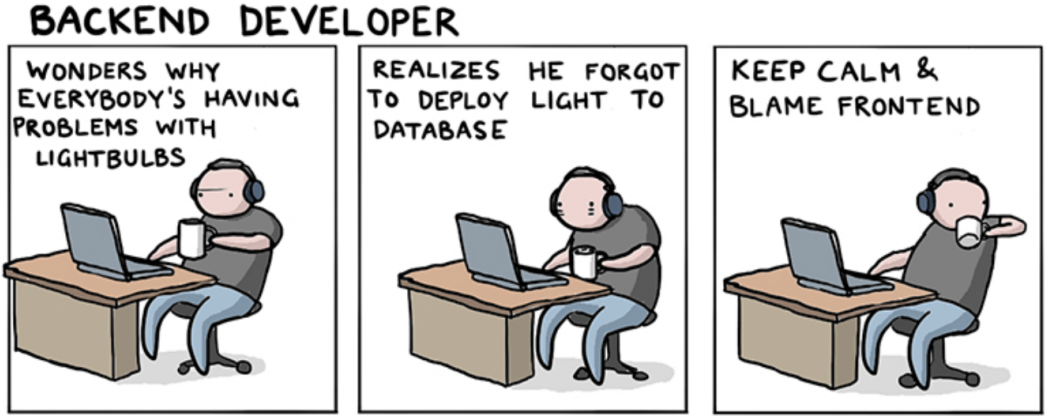
Basic Concepts

Key Concepts in a DBMS

- **Data**: Raw facts that can be recorded and have meaning.
- **Database**: A logically coherent collection of data designed for a specific purpose.
- **DBMS**: Software for defining, constructing, manipulating, and sharing databases.
- **Data Model**: Conceptual framework describing data, relationships, and constraints.

Purpose of Database Systems

The Purpose of using DBMS



The Purpose of using DBMS

The Purpose of using DBMS

In the early days, database applications were built directly on top of file systems, which leads to

- **Data redundancy and inconsistency:** data is stored in multiple file formats resulting invduplication of information in different files
- **Difficulty in accessing data:** Need to write a new program to carry out each new task
- **Data isolation:** Multiple files and formats
- **Integrity problems:**
  - Integrity constraints (e.g., account balance > 0) become “buried” in program code rather than being stated explicitly
  - Hard to add new constraints or change existing ones

The Purpose of using DBMS

- **Atomacity of Updates:**
  - Failures may leave database in an inconsistent state with partial updates carried out
  - Example: Transfer of funds from one account to another should either complete or not happen at all
- **Concurrent access by multiple users:**
  - Concurrent access needed for performance
  - Uncontrolled concurrent accesses can lead to inconsistencies. Example : Two people reading a balance (say 100) and updating it by withdrawing money (say 50 each) at the same time
- **Security Problems:** Hard to provide user access to some, but not all, data

Database systems offer solutions to all the above problems

View of Data

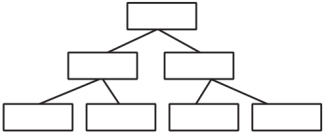
What do we mean by ”View of Data”?

A database system is a collection of interrelated data and a set of programs that allow users to access and modify these data. A major purpose of a database system is to provide users with an abstract view of the data.

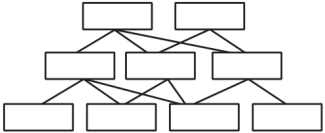
That is, the system hides certain details of how the data are stored and maintained.

What is a Data Model?

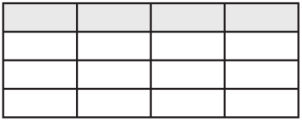
When you use the term database, what kind of database do you mean? There are many types available for data management. The data association and operation methods that a database uses is called its data model.



Hierarchical data model



Network data model



Relational data model

Underlying the structure of a database is the data model

Data Models Overview

- A **data model** defines how data is structured and related.
- It is a collection of tools for describing
  - Data
  - Data relationships
  - Data semantics
  - Data constraints
- Common types:
  - **Relational Model:** Data stored in tables.
  - **Entity-Relationship (ER) Model:** Used in conceptual design.
  - **Semi-structured Model:** XML, JSON data.
  - **Object-based Model:** Integrates object-oriented ideas.
  - **Other older models:** Network model, Hierarchical model
- Example: Table of **Students(ID, Name, Department)**.

Relational Model

Columns

Rows

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

A sample Relational Database

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

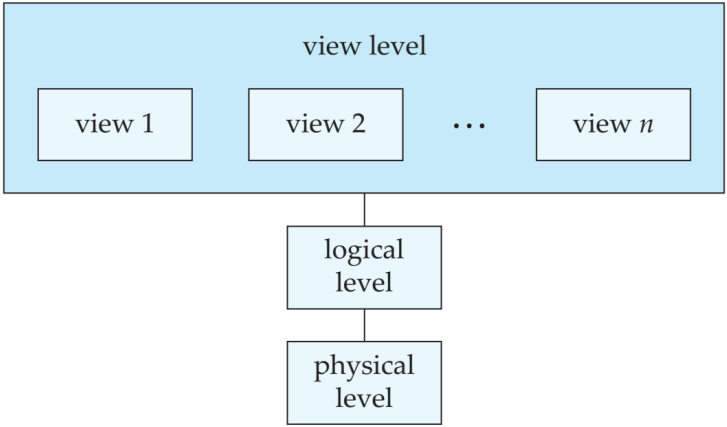
dept_name	*building*	*budget*
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The *department* table

Data Abstraction

- For the system to be usable, it must retrieve data efficiently.
- The need for efficiency has led database system developers to use complex data structures to represent data in the database.
- Since many database-system users are not computer trained, developers hide the complexity from users through several levels of data abstraction, to simplify users' interactions with the system:

Data Abstraction



Physical Level — How Data Are Stored

- **Lowest level** of abstraction.
- Describes the **actual storage** of data in memory blocks, files, and indexes.
- Involves byte-level details, record formats, and access paths.
- Example:
  - A table may be stored as **a sequence of records** in a file.
  - Attributes may be separated by commas, or stored with fixed lengths.
  - Variable-length fields may store the length followed by data.
- **Indexes** are used to retrieve data efficiently.

This level is invisible to most users and developers.

Logical Level — What Data Are Stored

- Represents the **entire database** using simple structures such as tables and records.
- Focuses on **data organization** and **relationships**.
- Provides **physical data independence** — changes at the physical level do not affect logical design.
- Used by **database administrators** and **developers**.

Example — Defining a Record Type

```
type instructor = record
ID          : char(5);
name        : char(20);
dept_name   : char(20);
salary      : numeric(8,2);
end;
```

Analogy: Just as a compiler hides memory layout in a programming language, a DBMS hides physical storage details from this logical view.

View Level — What Users See

- **Highest level** of abstraction.
- Each user sees only the **relevant portion** of the database.
- Simplifies access and provides **security control**.
- Example:
  - The registrar's office can view **student data** but not instructor salaries.
  - The finance office can view **salary information** but not student grades.
- One database can have **many views** for different applications.

Users at this level never see how or where data are stored.

Analogy: Programming Language vs. Database Levels

- **Physical Level** – Byte-level layout, files, indexes. **Hidden from programmers.**
- **Logical Level** – Record definitions and constraints. **Visible to developers.**
- **View Level** – Application-facing data shown to end users. **Used for security and simplicity.**

Programming Analogy

Database Concept	Programming Analogy
Physical Level	Byte layout of structs
Logical Level	Type definitions (records/classes)
View Level	GUI / API output to users

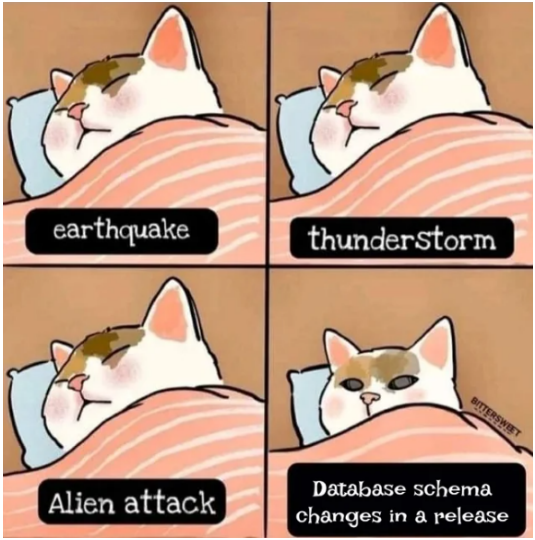
Schemas and Instances

Schemas

Similar to types and variables in programming languages

- **Schema:** The design or structure of a database (like a blueprint).
- **Logical Schema:**
  - What data is stored and how it's related.
  - The overall logical structure of the database.
  - Example: The database consists of information about a set of customers and accounts in a bank and the relationship between them.
  - Analogous to type information of a variable in a program
- **Physical schema:** The overall physical structure of the database

Schemas


earthquake
thunderstorm
Alien attack
Database schema changes in a release
BITTERSWEET

Instances

Similar to types and variables in programming languages

- **Instance:** the actual content of the database at a particular point in time. Analogous to the value of a variable
- **Physical Data Independence:** the ability to modify the physical schema without changing the logical schema
  - Applications depend on the logical schema
  - In general, the interfaces between the various levels and components should be well defined so that changes in some parts do not seriously influence others

Database Languages

Database Languages

- **DDL (Data Definition Language):** Defines the structure of the database.
  - Example: `CREATE TABLE Students (...);`
- **DML (Data Manipulation Language):** Used to access or modify data.
  - `INSERT, UPDATE, DELETE, SELECT`
- **SQL (Structured Query Language):**
  - Combines **DDL** and **DML**.
  - **Declarative:** Specify *what* you want, not *how* to get it.

DDL

Specification notation for defining the database schema
Example:

```
create table instructor (
ID char(5),
name varchar(20),
dept_name varchar(20),
salary numeric(8,2)
);
```

- DDL compiler generates a set of table templates stored in a data dictionary.
- Data dictionary contains metadata (i.e., data about data):
  - Database schema
  - Integrity constraints
  - Primary key (ID uniquely identifies instructors)
  - Authorization: Who can access what content

DML

Language for accessing and updating the data organized by the appropriate data model

- DML is also known as query language
- There are basically two types of data-manipulation language
  - Procedural DML – require a user to specify what data are needed and how to get those data.
  - Declarative DML – require a user to specify what data are needed without specifying how to get those data.
- Declarative DMLs are usually easier to learn and use than are procedural DMLs.
- Declarative DMLs are also referred to as non-procedural DMLs
- The portion of a DML that involves information retrieval is called a query language.

SQL

- SQL query language is nonprocedural. A query takes as input several tables (possibly only one) and always returns a single table.
- Example to find all instructors in Comp. Sci. dept:

```
select name
from instructor
where dept_name = 'Comp. Sci.';
```

- SQL is NOT a Turing machine equivalent language.
- To be able to compute complex functions, SQL is usually embedded in some higher-level language.
- Application programs generally access databases through one of:
  - Language extensions to allow embedded SQL.
  - Application program interfaces (e.g., ODBC/JDBC) which allow SQL queries to be sent to a database.

Database Design

Database Design

- A **database system** manages large bodies of information used by some **enterprise**.
- These data support an organization's products, services, or decisions.
- **Database design** mainly focuses on creating the **database schema**.
- A complete design also considers:
  - Application programs
  - User interfaces
  - Operational environment
- In this course/text: focus on **queries** and **schema design**; application design appears later (Chapter 9).

Database Design

- **Logical Design –** Deciding on the database schema. Database design requires that we find a “good” collection of relation schemas.
  - Business decision – What attributes should we record in the database?
  - Computer Science decision – What relation schemas should we have and how should the attributes be distributed among the various relation schemas?
- **Physical Design –** Deciding on the physical layout of the database

Database Engine

Database Engine

A database system is partitioned into modules that deal with each of the responsibilities of the overall system. The functional components of a database system can be divided into

- **Storage Manager:** Handles storage and retrieval of data.
- **Query Processor:** Parses and optimizes user queries.
- **Transaction Manager:** Ensures safe and consistent updates.
- These components work together for **efficient and reliable data management**.

Storage Manager

- A program module that provides the interface between the low-level data stored in the database and the application programs and queries submitted to the system.
- The storage manager is responsible to the following tasks:
  - Interaction with the OS file manager
  - Efficient storing, retrieving and updating of data
- The storage manager components include:
  - Authorization and integrity manager
  - Transaction manager
  - File manager
  - Buffer manager

Storage Manager

The storage manager implements several data structures as part of the physical system implementation:

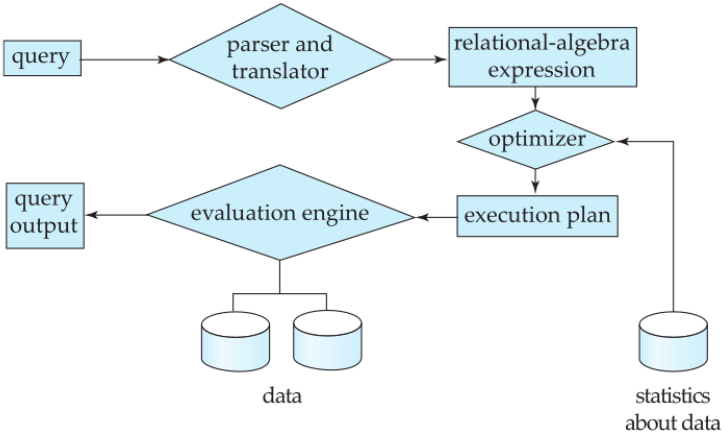
- Data files – store the database itself
- Data dictionary – stores metadata about the structure of the database, in particular the schema of the database.
- Indices – can provide fast access to data items. A database index provides pointers to those data items that hold a particular value.

Query Processor

- The query processor components include:
  - **DDL interpreter** – interprets DDL statements and records the definitions in the data dictionary.
  - **DML compiler** – translates DML statements in a query language into an evaluation plan consisting of low-level instructions that the query evaluation engine understands.
  - The DML compiler performs query optimization; that is, it picks the lowest cost evaluation plan from among the various alternatives.
- Query evaluation engine – executes low-level instructions generated by the DML compiler

Query Processing

1. Parsing and translation
2. Optimization
3. Evaluation



Transactions

- A **transaction** is a collection of operations that performs a single logical function in a database application
- **Transaction-management component** ensures that the database remains in a consistent (correct) state despite system failures (e.g.,power failures and operating system crashes) and transaction failures.
- **Concurrency-control manager** controls the interaction among the concurrent transactions, to ensure the consistency of the database.

Database Architecture

Database Architecture

- **Centralized databases:** One to a few cores, shared memory
- **Client-server:** One server machine executes work on behalf of multiple client machines.
- **Parallel databases:** Many core shared memory, Shared disk, Shared nothing
- **Distributed databases:** Geographical distribution, Schema/data heterogeneity

Application Architectures

Database-Driven Applications:

- Applications using databases can have:
  - **Two-tier architecture**
  - **Three-tier architecture**

Two-Tier Architecture:

- Used in earlier generations.
- The **application runs on the client machine**.
- Database functionality is accessed on the server through query statements.

Three-Tier Architecture

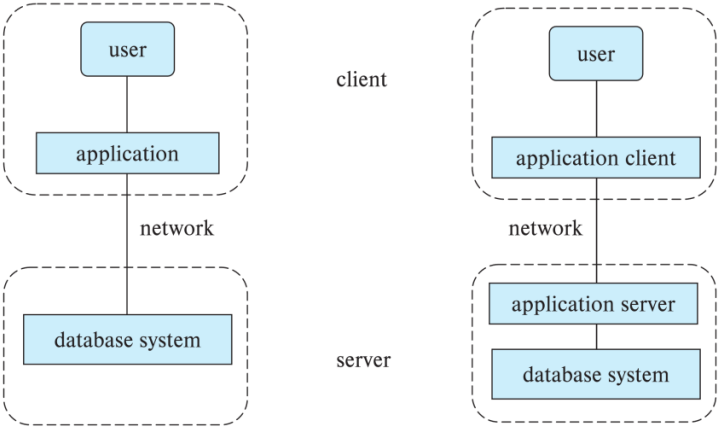
Modern Applications:

- Most modern systems use a **three-tier architecture**.
- The client (e.g., web browser, mobile app) acts as a **front end** only.
- The front end communicates with an **application server**.
- The application server, in turn, accesses the **database system**.

Advantages:

- Centralized **business logic** within the application server.
- Improved **security**.
- Enhanced **performance and scalability**.

Two-tier and three-tier architectures



(a) Two-tier architecture (b) Three-tier architecture

Database Users and Administrator

Database Users and Administrators

Goal of a Database System:

- To **retrieve information** from and **store new information** in the database.
- People who work with a database fall into two categories:
  - **Database Users**
  - **Database Administrators (DBAs)**

Database Users and User Interfaces

Four Types of Users:

1. **Naïve users**
2. **Application programmers**
3. **Sophisticated users**
4. **Specialized users**

Interaction Styles:

- Each user type interacts differently with the database.
- Therefore, different **user interfaces** are designed for each.

Naïve Users

Characteristics:

- Unsophisticated users.
- Use **predefined interfaces** such as web or mobile apps.
- Typical interface: a **forms-based interface**.

Example Scenario:

- A student registers for a class using a web interface.
- The system:
  1. Verifies user identity.
  2. Displays a registration form.
  3. Submits entered data to the server.
  4. The server checks for available seats and updates the class roster in the database.

Application Programmers

Who They Are:

- Computer professionals who write programs that interact with the database.
- Use tools and APIs to create user interfaces and connect with the database.

Key Responsibilities:

- Develop and maintain **application logic**.
- Implement secure and efficient data access methods.

Sophisticated Users

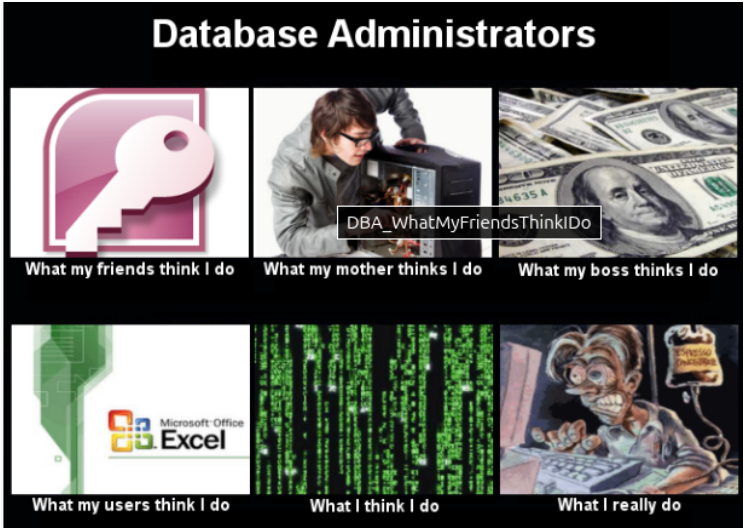
Characteristics:

- Interact with the system directly, without writing application code.
- Use **query languages** (e.g., SQL) or **data analysis tools**.

Examples:

- Data analysts exploring large datasets.
- Business intelligence professionals performing ad-hoc queries.

Database Administrator (DBA)



Database Administrator (DBA)

Definition:

- A person with **central control** over both data and the programs that access it.
- Ensures smooth operation, security, and performance of the database system.

Key Responsibilities:

1. **Schema definition**
2. **Storage structure and access-method definition**
3. **Schema and physical-organization modification**
4. **Authorization management**
5. **Routine maintenance**

Summary

- **DBMS** manages data efficiently and securely.
- Solves problems like **redundancy** and **inconsistency**.
- **SQL** is the core language for defining and querying data.
- **Transactions** ensure reliability through **ACID**.
- **Modern DBMSs** continue to evolve with technology.

Questions?